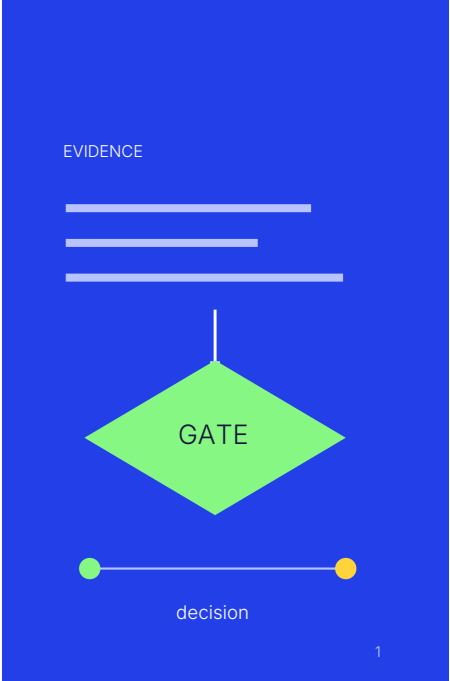
LAB 07 / WEEK 04

MLOps: Machine Learning Operations

Training Pipeline, Model Evaluation & Quality Gate

Automate a candidate, justify a decision and preserve the accepted model. Needs Lab 6 reviewed and merged on main.



Show three outcomes

A reviewer should find evidence for every claim.

1 **Automate**

One documented entry point produces a decided candidate from declared inputs in a clean checkout.

2 **Decide**

A policy committed before evaluation yields explainable decisions, including a genuine rejection.

3 **Preserve**

Rejection leaves accepted state unchanged; promotion retains the accepted version it replaces. Your Cloud Run service serves the accepted version; the previous revision stays available.

If nothing qualifies, retain the accepted model and report the actual outcome; do not force a pass.

Agree a contract, then divide the work

Rotate the lead; contribute and review another member's work.



A team of three or four may divide work differently; the contract remains shared.

Make the work discoverable

The review contract is fixed; your internal layout is your choice.

KEYWORDS + QUESTIONS

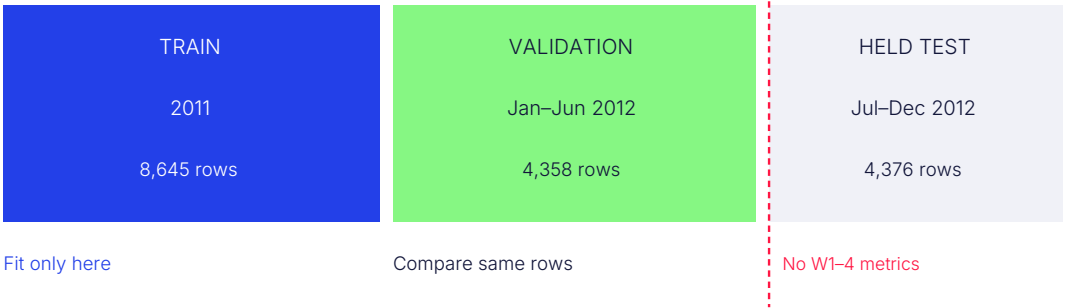
Link policy and decision evidence from the README. Keep model binaries and the tracking database out of Git.

README.md	add “Pipeline and gate”
policy	commit it; link its chosen location
decisions	commit records; link their location
team report	add a Lab 7 section

Document accepted-bundle recovery and the entry point a reviewer can run.

Keep the evaluation boundary unchanged

The frozen snapshot and twelve ordered predictors remain the project contract.



Candidates vary model settings only. Missing hours are valid; keep the canonical CSV unchanged.

Inspect the system you actually have

The published starter is a Week 1 scaffold; start from your team's completed code.

1 **Training boundary**

Read src/bike_demand/train.py and your completed implementation.

2 **Overwrite risk**

Which operation could replace a model before the gate decides?

3 **Settings guard**

Where are model settings constrained, and how will review preserve the data and feature contracts?

Do not assume a pipeline command, promotion API or registry already exists.

Design the pipeline boundary

Answer with a choice, an alternative and a reason.

KEYWORDS + QUESTIONS

Write the answers in the README before the implementation becomes difficult to change.

stage	what does it consume and produce?
entry point	what does a clean checkout need?
failure	what can safely be retried?
partial	what happens to incomplete output?
identity	which inputs determine this run?

One command is a reproducibility interface; its name and stage layout are yours.

Design a fair comparison

A score needs its evaluation context.

KEYWORDS + QUESTIONS

Compare with the train-mean baseline and a named accepted model. Explain how comparability is established.

reference	which accepted version is frozen?
observations	same rows and labels for both?
metric	what does the score mean, in what unit?
margin	what size of change matters, and why?
provenance	what makes an old score reusable?

Unavailable matching incumbent evidence cannot be claimed as passed.

Commit the rule before the candidate

The policy must not be chosen to fit an observed result.



Commit order and raw execution evidence support this claim.

The gate must defend three claims

Lecture 7's Data checks are part of Integrity here. Justify the rule form and thresholds.

Quality

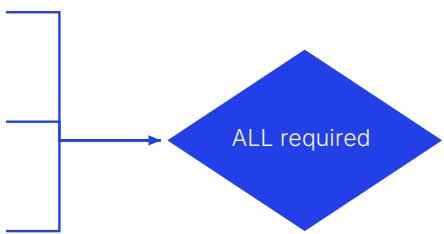
naive + accepted model; same validation

Operation

justified serving condition

Integrity

evaluated inputs + exact candidate bytes



A missing check blocks eligibility.

Every mandatory criterion must pass; metric improvement cannot replace missing evidence.

Decide what failure means

A measured rejection differs from an incomplete evaluation.

KEYWORDS + QUESTIONS

The decision record explains why promotion is allowed or prevented.

invalid	what evidence must never count as pass?
missing	what if a required check did not run?
reject	which measured rule failed?
blocked	what must be repaired before deciding?
status	how does CI observe the outcome?

Unavailable checks are not passed checks. Preserve the reasons.

Build in reviewable increments

Use the environment lock, tests and code already established.

1 **Known evidence**

Establish the accepted baseline and a reproducible local execution path.

2 **Connected decision**

Make the chosen stages produce evidence for the committed policy.

3 **Independent challenge**

Have a reviewer demonstrate a real failure and inspect what changed.

Do not mute checks or edit evidence to make an outcome look successful.

Design the accepted-state boundary

Candidate work must not silently become serving state.

KEYWORDS + QUESTIONS

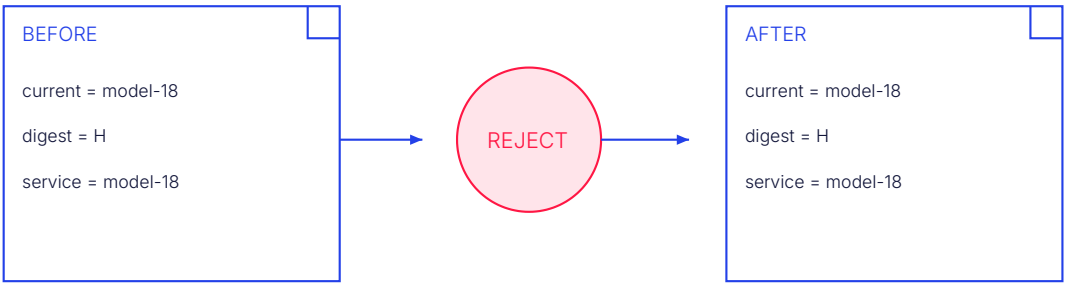
Explain the failure behavior as well as the successful path.

separate	what may a candidate write?
identity	what stays immutable for its lifetime?
switch	what if promotion stops halfway?
concurrent	what if the incumbent changed?
retain	what recovers without retraining?

The layout, reference mechanism and promotion command are team decisions.

Prove that rejection preserves state

A decision message alone is not enough evidence.

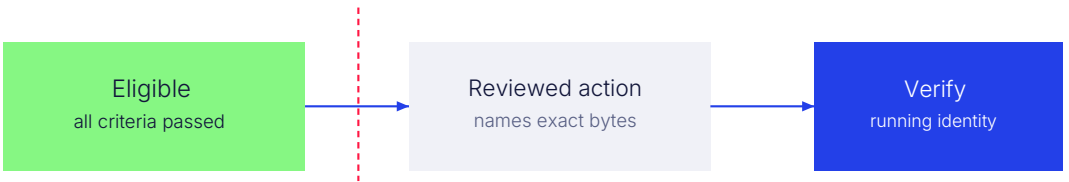


The decision log changes. The accepted bundle does not.

Use your own trusted artifacts; do not load an unknown model to create a fault.

An honest accept has a reason

Better MAE is not guaranteed under the fixed project contract.



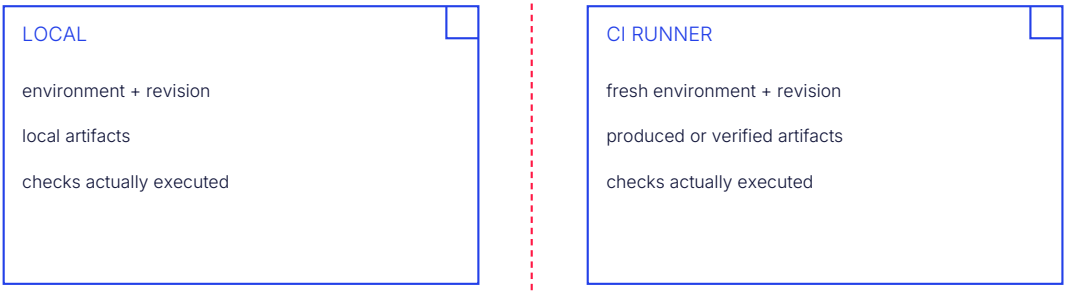
Retain the accepted version this action replaces.

An accepted equivalent does not demonstrate improved accuracy.

If nothing qualifies, retain the accepted model and report what is incomplete.

Separate local and CI claims

A clean runner does not inherit laptop models or the local MLflow store.



Evidence does not transfer by assumption.

Promotion is manual from reviewed main. CI training is optional; serving on Cloud Run is required.

Review a claim you did not implement

The reviewer chooses a meaningful challenge and retains the result.



Predict

Which claim should the challenge expose?

Observe

Inspect the decision, artifacts and served state.

Explain

Record the gap; repair its cause through review.



Green without the intended check running is a gap, not a successful challenge.

Optional: improve one measured trade-off

A bounded experiment can strengthen the engineering explanation.

1 **Hypothesis**

Use validation error analysis to predict what a model-setting change should improve.

2 **Budget**

Declare a small candidate or compute budget and retain every attempt.

3 **Comparison**

Compare quality and an operating cost, then justify promotion or retention.

No extra grade component and no requirement to obtain a better score.

Keep evidence that survives the session

Raw records support the claim; screenshots supplement them.

Provenance

Policy revision, identities, evaluation context, reasons

State change

Rejected-state comparison, approval, Cloud Run revision, served identity

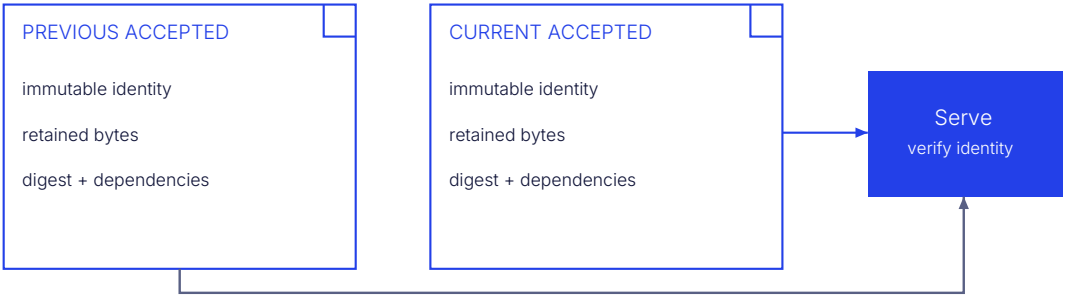
Reproduce

README entry point, recovery location, limitations

Save raw text in the repository or report; use at most three screenshots.

Leave a recoverable input for Lab 8

The next lab uses the versions and evidence preserved here.



Each accepted version needs a Cloud Run revision. Report an incomplete handoff before the drill.

m4 joins both Week 4 labs

Assessment remains exactly the syllabus assessment.

1 **Team checkpoint m4**

Tag link and note after Labs 7–8: gate decisions, incident diagnosis and verified rollback. Ungraded.

2 **Individual report**

reports/week-4.md in your private report repository: contributions, evidence, problems and learning. Worth 10%.

3 **Final team release**

Week 5 clinic, verification and presentations; GitHub Release v1.0.0. Team project and presentation: 60%.

Four reports total 40%. No Week 5 individual report. Use the published deadline and time zone.

EXIT CHECK

Could a rejected model replace the accepted one? Show the evidence.

Keep the decision, preserve the accepted version and explain what your gate still cannot know.